

Barath Kumar

Senior Software Engineer



(945) 234-1431



barathkumar6610@outlook.com

PROFESSIONAL SUMMARY

Senior Software Engineer with over 10 years of experience delivering cloud-native, full stack solutions across finance, healthcare, and retail industries. Expert in building reactive, high-performance **Java microservices**, designing efficient **React.js, Angular** UIs and ensuring reliability through **multithreading** and advanced **exception handling**. Proven ability to drive automation and real-time processing with **Kafka, LLMs**, and **AI-enhanced decisioning**, while deploying secure, scalable platforms using **AWS, Azure**, and modern **DevOps** practices. Skilled in building and managing data platforms using **Databricks** for large-scale transformations and **Snowflake** for high-performance analytics. Experienced in leading end-to-end SDLC in **Agile environments**, collaborating with cross-functional teams, and implementing **CI/CD** pipelines for reliable releases. Strong background in observability, performance tuning, and system resilience to support high-availability applications. Skilled in **API security, RBAC**, data governance, and managing hybrid architectures with **Docker**, and **Kubernetes**. Demonstrated ability to integrate third-party systems, optimize development velocity, and align engineering output with business goals through automation, monitoring, and continuous learning.

CERTIFICATIONS

AWS Certified: Solutions Architect Professional | Credential: 5b613d99606c4760b3331bf4adc3272d

Microsoft Certified: Azure AI Engineer Associate | Credential: F22C6B08F3364EC0

SKILLS

- **Languages & Frameworks:** Java 17/11/8/1.6, Spring Boot, Spring WebFlux, Spring MVC, Python, Node.js, Express.js, HTML, CSS, JavaScript, TypeScript, Django
- **Frontend Technologies:** React.js, Angular, Redux, React Hooks, Lifecycle Methods, Swagger/OpenAPI
- **Cloud & DevOps:** AWS (EKS, Lambda, EC2, S3, Secrets Manager, IAM), Azure (AKS, VNet, Key Vault), Terraform, GitLab CI/CD, Jenkins
- **Security & Auth:** OAuth 2.0, Spring Security, RBAC
- **Data & Streaming:** Kafka, Kafka Streams, RabbitMQ, PostgreSQL, MongoDB, Redis, Oracle, PL/SQL, Spring Batch, Snowflake, Databricks
- **AI & ML:** Generative AI, LLMs, Prompt Engineering, RAG architecture, Deep Learning, Scikit-learn, AWS Sagemaker, AWS Bedrock.
- **Monitoring & Testing:** New Relic, Splunk, Spring Actuator, JUnit, Mockito, PyTest, SonarQube, SLF4J
- **Architecture & Methodologies:** Microservices, RESTful APIs, Agile(Scrum), CI/CD Pipelines, GitOps

PROFESSIONAL EXPERIENCE

SENIOR SOFTWARE ENGINEER

Citi Bank, Irving, TX.

Jan 2023–Present

At Citi Bank modernized its servicing ecosystem for high-volume co-branded credit programs to address fragmented data flows, delayed dispute handling, inconsistent reward reconciliation, and limited visibility into credit line adjustments. A centralized platform was introduced to unify transaction processing, automate dispute resolution, align rewards accurately, and manage real-time credit updates—enhancing financial precision, operational control, and compliance across partner channels.

- Developed backend microservices using **Java 17**, **Spring Boot**, and **Spring WebFlux**, leveraging **reactive programming** to build non-blocking, event-driven services for real-time transaction processing, dispute lifecycle orchestration, credit line adjustments, and reward reconciliation at scale.
- Designed and documented secure **RESTful APIs** with **Swagger/OpenAPI**, enabling integration with internal platforms and external partner ecosystems for dispute submission, transaction review, and reward status.
- Secured APIs with **OAuth 2.0** and **Spring Security**, supporting fine-grained authorization for dispute handlers, partner systems, and customer service agents.
- Routed all external and internal service traffic through **Amazon API Gateway**, enforcing throttling, token validation, and centralized monitoring for dispute, reward, and credit service endpoints.
- Applied **ExecutorService** and thread pooling for parallel processing of reward reconciliations and bulk credit evaluations, significantly reducing turnaround times during high-traffic cycles.
- Cached dispute states, reward histories, and transaction flags in **Redis** to support responsive multi-step workflows and reduce load on downstream systems.
- Instrumented all services using **Spring Boot Actuator**, exposing health checks, dependency graphs, thread metrics, and custom business indicators for centralized observability.
- Streamed real-time transaction, dispute, and credit line events via **Kafka**, enabling asynchronous workflows and downstream enrichment using **Kafka Streams**.
- Built data transformation and enrichment modules in **Java**, storing structured records in **PostgreSQL** for downstream reconciliation, audit readiness, and partner-level reporting.
- Managed flexible document schemas in **MongoDB**, accommodating partner-specific formats for reward confirmations, credit memos, and dispute attachments.
- Deployed containerized microservices using **Docker** and orchestrated them with **Amazon EKS**, enabling scalable, isolated execution of credit, dispute, and reward servicing workflows across cloud environments.
- Provisioned and managed cloud infrastructure using **Terraform**, enabling reproducible, version-controlled deployments of Kubernetes clusters, S3 buckets, IAM roles, and Lambda triggers.
- Implemented **Blue-Green Deployment** strategies using Kubernetes and GitLab CI/CD to reduce downtime and mitigate deployment risks during servicing system upgrades.
- Automated event-driven tasks with **AWS Lambda** for functions such as reward recalculations, credit limit notifications, and chargeback processing tied to S3 and API events.
- Handled document uploads, dispute evidence, and credit audit files through **Amazon S3**, enforcing access control, lifecycle policies, and versioned storage.
- Secured environment variables, tokens, and credentials using **AWS Secrets Manager**, supporting secure, audit-compliant secret rotation across distributed microservices.
- Developed interactive servicing dashboards using **Node.js**, **NPM** and **React.js**, incorporating **Redux** for state management, **GraphQL** for efficient data querying, and **lazy loading** techniques to enhance performance across high-volume, data-intensive views.
- Wrote and executed unit tests for all UI components using **Jest**, ensuring functional stability and regression coverage for React-based credit servicing modules.

- Integrated **LLM-based services** using **AWS Bedrock** and **RAG architecture** to parse financial documents, detect anomalies, and augment agent responses with policy-aligned insights.
- Trained and deployed ML models in **AWS SageMaker** to score risk patterns, validate dispute claims, and detect inconsistent reward allocations at scale.
- Centralized monitoring with **New Relic** and **AWS CloudWatch**, visualizing service health, SLA compliance, and throughput metrics across all servicing domains.
- Enforced release quality using **GitLab CI/CD**, static analysis with **SonarQube**, and coverage gates for mission-critical servicing modules.
- Validated core APIs and data workflows using **JUnit**, **PyTest**, and **Postman** collections to test boundary cases, contract integrity, and scenario-based flows.
- Provided 24/7 on-call support through **PagerDuty**, triaging production issues tied to dispute resolution SLAs and transaction posting delays.
- Built end-to-end **ETL** pipelines using **Databricks** for transaction data transformation and **Snowflake** for structured reporting, enabling large-scale joins, schema evolution, and compliance tracking across partner programs.
- Participated in **Agile** ceremonies and sprint delivery using **Jira**, collaborating with product, compliance, and partner teams to deliver servicing improvements across domains.
- Improved dispute resolution turnaround by 40%, reduced reward calculation errors by 65%, and enhanced credit line adjustment accuracy through automation
- Collaborated with **cross-functional teams** including fraud analysts, data scientists, and product managers to align model outputs and servicing workflows.

Environment: *Java 17, Spring Boot WebFlux, Kafka Streams, React.js, AWS EKS, Redis, MongoDB, Databricks, AWS Lambda, Snowflake, SageMaker, Terraform, GitLabCI/CD, AWS Secrets Manager, CloudWatch, Postman Relic, Postman*

SOFTWARE ENGINEER

GAF, Parsippany, NJ.

Jul 2019–Dec 2022

At GAF, developed a data-driven intelligence platform to monitor vendor-level product consumption across its B2B distribution network. The solution provided real-time insights into purchasing behavior, highlighting which SKUs were most frequently ordered, how usage patterns varied by region and vendor type, and when anomalies like sudden demand shifts occurred. By integrating with order and sales systems, it enabled automated tracking, vendor segmentation, and feedback analysis, allowing GAF to improve demand forecasting, tailor vendor engagement strategies, and optimize inventory planning with greater precision and responsiveness.

- Built backend services using **Java 11** and **Spring Boot** to collect and expose vendor usage metrics, track SKU-level consumption, and support real-time demand analytics.
- Developed microservices to process transactional data, classify vendor behavior patterns, and automate insights delivery through **RESTful APIs**.
- Applied **Resilience4j** circuit breakers with fallback strategies to ensure availability and graceful degradation during vendor analytics ingestion and reporting.
- Enabled asynchronous, event-driven workflows using **Kafka** to capture order activities, detect anomalies, and trigger escalation logic based on usage signals.
- Built secure, tokenized APIs with **OAuth 2.0** across microservices to control access to sensitive vendor and product intelligence within Azure environments.
- Designed safe retry and idempotent logic using **Java** to handle duplicate order streams and ensure consistency in usage reporting.
- Deployed external APIs for vendor engagement metrics on **Azure Kubernetes Service (AKS)**, enabling scalable access from analytics tools and BI platforms.
- Provisioned internal analytics dashboards and portals using **AKS, Azure Active Directory(Entra ID)**, and segmented **VNet** configurations for secure, role-based data access.

- Managed credential storage and rotation using **Azure Key Vault**, securing access to vendor ingestion pipelines and usage datasets.
- Configured **GitLab CI/CD pipelines** to automate testing, validation, and secure deployment of microservices, integrating static code analysis via **SonarQube**.
- Implemented in-memory caching with **Spring Cache** and **Redis** to accelerate lookups for trending SKUs, vendor activity, and region-specific analytics.
- Stored structured transactional and product usage data in **Azure SQL Database**, supporting analytics, reporting, and SLA-bound queries.
- Managed unstructured feedback, documents, and behavioral signals in **Azure Cosmos DB**, allowing for schema flexibility and global distribution.
- Stored structured purchase data in **PostgreSQL** and unstructured engagement logs in **MongoDB**, enabling downstream analytics and audit trail retention.
- Built **ETL pipelines** using **Apache NiFi** to ingest vendor transaction logs and feedback files, validate schema integrity, and persist enriched data into MongoDB.
- Automated batch ingestion and archival workflows with **Control-M**, generating periodic SLA-compliant reports and verifying pipeline health.
- Developed internal dashboards using **Node.js, NPM and Angular (MEAN Stack)** to visualize vendor-level product usage, reorder frequency, and sales velocity trends.
- Applied **Angular services** and **RxJs** to manage real-time updates and behavioral widgets in consumption monitoring dashboards.
- Integrated **deep learning models** built in **Python** to auto-segment vendors by usage intensity, category affinity, and risk factors, reducing manual profiling workload.
- Created observability dashboards with **Splunk** to monitor ingestion rates, queue depths, and system anomalies across the data pipeline.
- Routed structured logs via **SLF4J and Logback** into **Splunk**, enabling trace-level tracking of vendor activity across services.
- Delivered product usage KPIs to **Power BI**, helping business teams identify top-performing SKUs, churn risks, and engagement opportunities.
- Ensured stability and correctness with unit and integration testing using **JUnit** and **Mockito**; validated API contracts and edge cases using **Postman** during regression cycles
- Developed ETL workflows in **Databricks** to clean and enrich high-frequency vendor order data, enabling fulfillment trend analysis, SLA tracking, and anomaly detection.
- Operated within an **Agile** delivery model using **Azure Boards** for sprint planning, backlog grooming, and cross-functional coordination.
- Worked with **cross-functional teams** across vendor ops, supply chain, and analytics to define metrics and validate business insights

Environment: Java 11, Spring Boot, Kafka, Angular, Azure Kubernetes Service (AKS), Azure SQL Database, Azure Cosmos DB, Azure Key Vault, Apache NiFi, Control-M, Redis, RabbitMQ, GitLab CI/CD, SonarQube, OAuth 2.0, Power BI, Splunk, Python, JUnit, Mockito, Postman, RxJs, Azure Active Directory, Azure Boards

SOFTWARE ENGINEER

Apollo Hospitals, Hyderabad, India.

Dec 2017–Jun 2019

At Apollo Hospitals, designed and implemented a scalable system to process high volumes of patient data, handle real-time admissions, and manage dynamic case intake across departments. The solution featured secure integration with external insurance authorization services to automate eligibility checks and approval workflows, reducing manual intervention. Leveraging event-driven architecture and role-based access controls, it enabled seamless data flow, improved system responsiveness, and enhanced coordination among clinical, administrative, and billing units.

- Developed scalable backend services with **Spring Boot, Java 8, and Node.js** to handle real-time patient data

ingestion, eligibility checks, and clinical task orchestration.

- Migrated monolithic Java applications built on **Spring MVC** to modular **Spring Boot microservices**, improving maintainability, scalability, and deployment agility across core clinical and billing workflows
- Integrated external APIs securely for insurance eligibility validation, applying **API security best practices** with token-based authentication and input sanitation.
- Applied **Java 8** functional programming features (streams, lambdas) to reduce boilerplate in data transformation modules used for dynamic case routing.
- Automated deployment and scaling of services on **AWS EC2 instances**, creating environment-specific build pipelines using **Git** and **CI/CD** practices under Agile delivery.
- Enabled secure document upload and retrieval of prescriptions and insurance documents via **Amazon S3**, applying **IAM roles** and pre-signed URLs for access control.
- Automated batch ingestion of insurance claims and authorization logs into **Amazon S3** using **AWS Data Pipeline**, enabling downstream analytics and archival in compliance with hospital data retention policies.
- Used **SonarQube** for static code analysis and security rule enforcement across backend microservices, maintaining enterprise-grade quality gates for all commits
- Designed interactive, role based front-end dashboards using **Angular**, and **Bootstrap** to visualize patient intake queues, case progression, and insurance status across departments.
- Implemented asynchronous communication between modules using **RabbitMQ**, ensuring low-latency updates for insurance approvals, billing triggers, and room assignments.
- Optimized database operations using **MySQL** stored procedures and indexes, improving data retrieval speed for high-volume patient queries and treatment history access.
- Developed predictive scripts in **Python** using **Scikit-learn** to analyze historical patient data and forecast ER peak hours, helping staff prepare resource allocation in advance.
- Built internal admin tools using **Django** to monitor data flow, trigger retries on failed integrations and allow manual override of insurance decision flags.
- Configured **AWS CloudWatch alarms** and dashboards to monitor EC2 instance health, REST API latency, and RabbitMQ message throughput, enabling proactive issue detection and SLA compliance for patient-critical workflows.
- **JUnit** regression testing for end-to-end clinical and billing workflows. Delivered sprint-based updates in **Agile** teams, performing

Environment: Java 8, Spring Boot, Node.js, Angular, AWS EC2, Amazon S3, RabbitMQ, MySQL, Scikit-learn, Django, Git CI/CD

SOFTWARE ENGINEER

BigBasket, Bengaluru India.

Feb 2015– Nov 2017

At BigBasket, I worked on enhancing the e-commerce platform to improve both operational efficiency and customer engagement. I developed features for real-time inventory tracking, optimized order workflows, and implemented personalized recommendation modules to enrich the user experience. Additionally, I contributed to marketing automation tools that leveraged user behavior data to deliver targeted campaigns, driving higher conversions and repeat purchases.

- Developed order management and checkout modules using **Java 1.6** and **Spring MVC**, ensuring accurate cart validation and price calculations.
- Built **RESTful APIs** for dynamic product listings, filtering, and availability checks to support high-volume customer interactions.
- Developed batch data pipelines using **Java** to sync real-time stock data from vendor APIs, applying validation and transformation before loading into **Oracle** for inventory tracking and analytics
- Designed promotional rule engines to manage flash sales and discounts, integrating with catalog services via **Spring**.
- Deployed Java applications on **Apache Tomcat** servers, managing WAR files and tuning performance for

peak demand periods.

- Automated nightly builds and deployments using **Apache Maven** and **shell scripts** to ensure reliable code promotion.
- Wrote optimized **PL/SQL** queries in **Oracle** to generate category-level reports, reduce query response time, and support analytics needs.
- Created admin tools using **HTML**, **CSS**, and **JavaScript** for warehouse staff to monitor low stock and delayed shipments.
- Used **Apache ActiveMQ** for event-driven communication to handle stock updates, order notifications, and cart expiration workflows.
- Authored unit tests using **JUnit**, improving code stability and reducing post-deployment issues during seasonal sale spikes.

Environment: *Java 1.6, Spring MVC, Oracle, Apache Tomcat, Apache ActiveMQ, Apache Maven, HTML/CSS/JavaScript*